



24. 圖論

2026資訊研究社算法班
Made with ❤️ by jheanlee

Leetcode 695 - Max Area of Island

有一個 $m \times n$ 的矩陣, 其中 1 代表陸地, 0 代表水。每個位置由上下左右四個方向連接。一個島嶼由互相鄰接的陸地(1) 組成。島嶼的面積為所有相連接的1 數量。

矩陣的邊界外都是水。

求矩陣中最大的島嶼面積。

Example 1:

0	0	1	0	0	0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	1	1	1	0	0	0
0	1	1	0	1	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	0	1	0	1	0	0
0	1	0	0	1	1	0	0	1	1	1	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	1	1	1	0	0	0
0	0	0	0	0	0	0	1	1	0	0	0	0

Input: grid = [[0,0,1,0,0,0,0,1,0,0,0,0,0],[0,0,0,0,0,0,0,0,1,1,0,0,0],[0,1,1,0,1,0,0,0,0,0,0,0,0],[0,1,0,0,1,1,0,0,1,0,1,0,0],[0,1,0,0,1,1,0,0,1,1,0,0],[0,0,0,0,0,0,0,0,0,0,1,0,0],[0,0,0,0,0,0,1,1,0,0,0],[0,0,0,0,0,0,0,1,0,0,0,0]]

Output: 6

Explanation: The answer is not 11, because the island must be connected 4-directionally.

Leetcode 695 - Max Area of Island

Example 1:

0	0	1	0	0	0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	1	1	1	0	0	0
0	1	1	0	1	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	0	1	0	1	0	0
0	1	0	0	1	1	0	0	1	1	1	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	1	1	1	0	0	0
0	0	0	0	0	0	0	1	1	0	0	0	0

Input: grid = [[0,0,1,0,0,0,0,1,0,0,0,0,0],[0,0,0,0,0,0,0,0,1,1,1,0,0],[0,1,1,0,1,0,0,0,0,0,0,0,0],[0,1,0,0,1,1,0,0,1,0,1,0,0],[0,1,0,0,1,1,0,0,1,1,1,0,0],[0,0,0,0,0,0,0,0,0,0,0,1,0],[0,0,0,0,0,0,0,1,1,1,0,0,0],[0,0,0,0,0,0,0,0,0,0,1,1,0,0,0]]

Output: 6

Explanation: The answer is not 11, because the island must be connected 4-directionally.



Leetcode 695 - Max Area of Island (DFS)

```
class Solution {
private:
    int island_area(vector<vector<int>> &grid, int i, int j) {
        if (i < 0 || i >= grid.size() || j < 0 || j >= grid[0].size()) return 0; // 檢查邊界
        if (grid[i][j] == 0) return 0; // 檢查是否為海水 or 處理過的島嶼
        grid[i][j] = 0; // 處理過的島嶼改成 0 避免重複計算

        return 1 // 記得現在這一格也是一個島嶼面積
            + island_area(grid, i + 1, j) // 下
            + island_area(grid, i - 1, j) // 上
            + island_area(grid, i, j + 1) // 右
            + island_area(grid, i, j - 1); // 左
    }
}
```



```
class Solution {
private:
    int island_area(vector<vector<int>> &grid, int i, int j) {
        if (i < 0 || i >= grid.size() || j < 0 || j >= grid[0].size()) return 0; // 檢查邊界
        if (grid[i][j] == 0) return 0; // 檢查是否為海水 or 處理過的島嶼
        grid[i][j] = 0; // 處理過的島嶼改成 0 避免重複計算

        return 1 // 記得現在這一格也是一個島嶼面積
            + island_area(grid, i + 1, j) // 下
            + island_area(grid, i - 1, j) // 上
            + island_area(grid, i, j + 1) // 右
            + island_area(grid, i, j - 1); // 左
    }

public:
    int maxAreaOfIsland(vector<vector<int>> &grid) {
        int res = 0;
        for (int i = 0; i < grid.size(); i++) {
            for (int j = 0; j < grid[0].size(); j++) {
                if (grid[i][j] == 1) {
                    res = max(res, island_area(grid, i, j));
                }
            }
        }

        return res;
    }
};
```

Leetcode 695 - Max Area of Island (BFS)

```
if (grid[i][j] == 1) { // 遇到沒處理過的島嶼
    int curr_area = 0;
    queue<pair<int, int>> q;
    q.emplace(x: [&i, y: [&j]);

    while (!q.empty()) {
        pair<int, int> curr = q.front();
        q.pop();

        // 檢查邊界
        if (curr.first < 0
            || curr.first >= grid.size()
            || curr.second < 0
            || curr.second >= grid[0].size()
        ) continue;
```

```
// 檢查是否為海水 or 處理過的島嶼
if (grid[curr.first][curr.second] == 0) continue;

curr_area++; // 增加島嶼面積

grid[curr.first][curr.second] = 0; // 處理過的島嶼改成 0 避免重複計算

// 把上下左右的格子放進queue裡
q.emplace(x: curr.first + 1, y: [&curr.second);
q.emplace(x: curr.first - 1, y: [&curr.second);
q.emplace(x: [&curr.first, y: curr.second + 1);
q.emplace(x: [&curr.first, y: curr.second - 1);
    }

    res = max(a: res, b: curr_area);
}
```

```

class Solution {
public:
    int maxAreaOfIsland(vector<vector<int>>& grid) {
        int res = 0;
        for (int i = 0; i < grid.size(); i++) {
            for (int j = 0; j < grid[0].size(); j++) {
                if (grid[i][j] == 1) { // 遇到沒處理過的島嶼
                    int curr_area = 0;
                    queue<pair<int, int>> q;
                    q.emplace(x: [&i, y: [&j]);

                    while (!q.empty()) {
                        pair<int, int> curr = q.front();
                        q.pop();

                        // 檢查邊界
                        if (curr.first < 0
                            || curr.first >= grid.size()
                            || curr.second < 0
                            || curr.second >= grid[0].size()
                        ) continue;

```

```

// 檢查是否為海水 or 處理過的島嶼
if (grid[curr.first][curr.second] == 0) continue;

curr_area++; // 增加島嶼面積

grid[curr.first][curr.second] = 0; // 處理過的島嶼改成 0 避免重複計算

// 把上下左右的格子放進queue裡
q.emplace(x: curr.first + 1, y: [& curr.second);
q.emplace(x: curr.first - 1, y: [& curr.second);
q.emplace(x: [& curr.first, y: curr.second + 1);
q.emplace(x: [& curr.first, y: curr.second - 1);
}

res = max(a: res, b: curr_area);
}
}

return res;
};

```



Leetcode 200 - Number of Islands

有一個 $m \times n$ 的矩陣, 其中 1 代表陸地, 0 代表水。每個位置由上下左右四個方向連接。一個島嶼由互相鄰接的陸地(1) 組成。

矩陣的邊界外都是水。

回傳所有的島嶼數量

Example 1:

```
Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
```

```
Output: 1
```

Leetcode 200 - Number of Islands (DFS)

```
class Solution {
public:
    void flood_island(vector<vector<char>>& grid, int i, int j) {
        if (j < 0
            || j >= grid[0].size()
            || i < 0
            || i >= grid.size())
            return;

        if (grid[i][j] == '0') return;

        grid[i][j] = '0';
        flood_island(grid, i + 1, j);
        flood_island(grid, i - 1, j);
        flood_island(grid, i, j + 1);
        flood_island(grid, i, j - 1);
    }
}
```

```
int numIslands(vector<vector<char>>& grid) {
    int cnt = 0;

    for (int i = 0; i < grid.size(); i++) {
        for (int j = 0; j < grid[0].size(); j++) {
            if (grid[i][j] == '1') { // 新的島嶼
                cnt++; // 數量++
                flood_island(grid, i, j); // 把這島嶼淹了 (避免重複計算)
            }
        }
    }

    return cnt;
};
```

Leetcode 200 - Number of Islands (BFS)

```
if (grid[i][j] == '1') { // 新的島嶼
    cnt++; // 數量++
    queue<pair<int, int>> q;
    q.emplace(x: [&i, y: [&j]);
    while (!q.empty()) { // 把這島嶼淹了 (避免重複計算)
        pair<int, int> curr = q.front();
        q.pop();

        if (curr.first < 0
            || curr.first >= grid.size()
            || curr.second < 0
            || curr.second >= grid[0].size()
        ) continue;

        if (grid[curr.first][curr.second] == '0') continue;

        grid[curr.first][curr.second] = '0';
        q.emplace(x: curr.first + 1, y: [&curr.second);
        q.emplace(x: curr.first - 1, y: [&curr.second);
        q.emplace(x: [&curr.first, y: curr.second + 1);
        q.emplace(x: [&curr.first, y: curr.second - 1);
    }
}
```

```

class Solution {
public:
    int numIslands(vector<vector<char>>& grid) {
        int cnt = 0;

        for (int i = 0; i < grid.size(); i++) {
            for (int j = 0; j < grid[0].size(); j++) {
                if (grid[i][j] == '1') { // 新的島嶼
                    cnt++; // 數量++
                    queue<pair<int, int>> q;
                    q.emplace(x: [&i, y: [&j]);
                    while (!q.empty()) { // 把這島嶼淹了 (避免重複計算)
                        pair<int, int> curr = q.front();
                        q.pop();

                        if (curr.first < 0
                            || curr.first >= grid.size()
                            || curr.second < 0
                            || curr.second >= grid[0].size()
                        ) continue;

```

```

                            if (grid[curr.first][curr.second] == '0') continue;

                            grid[curr.first][curr.second] = '0';
                            q.emplace(x: curr.first + 1, y: [&curr.second);
                            q.emplace(x: curr.first - 1, y: [&curr.second);
                            q.emplace(x: [&curr.first, y: curr.second + 1);
                            q.emplace(x: [&curr.first, y: curr.second - 1);
                        }
                    }
                }
            }

            return cnt;
        }
    };

```

2025上學期資研社期末賽 G. 迷路

<https://codeforces.com/group/nPd9lxKliH/contest/659833/problem/G>

你和 jnl 相約去南投大巨蛋看演唱會，但因為山路彎來彎去，jnl 迷路了。雖然手上有手機，但 jnl 是路癡，心急的他停下腳步打電話向你求助。

你打開了地圖軟體，地圖可以簡化成一個 $n \times n$ 的矩陣 m ，道路以 1 表示、走不了的樹林以 0 表示，每一步可以向 $+i$ 、 $-i$ 、 $+j$ 或 $-j$ 方向移動一格。你發現自己目前在 (i_0, j_0) ，而 jnl 告訴你他在 (i_1, j_1) 。

演唱會要開始了，請找出最短路徑共要走幾步，安撫他的心情。若不存在一個合法的路徑，請回傳 -1 並拋棄 jnl，等演唱會結束再去找他。

註：以直向為 i ，橫向為 j ，座標由 0 起算

Input

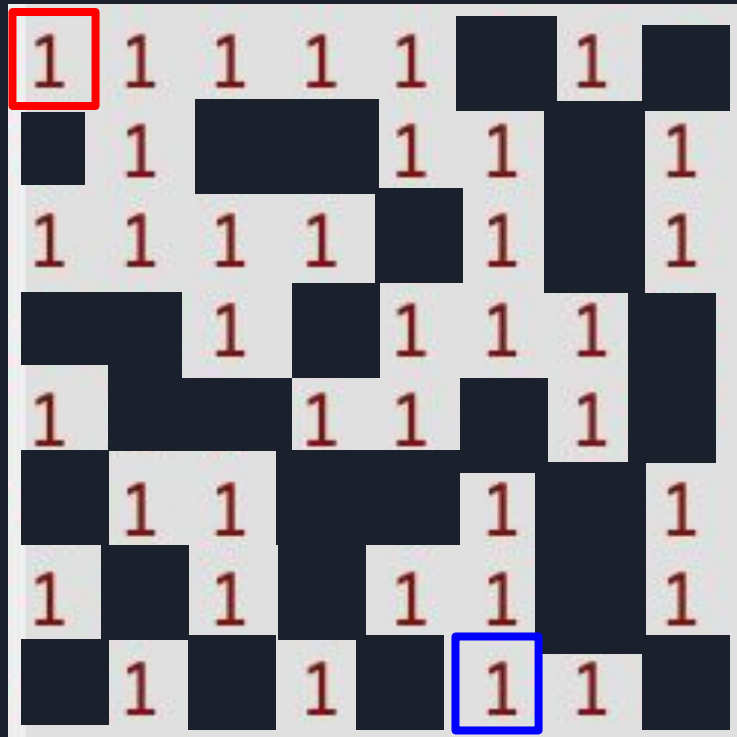
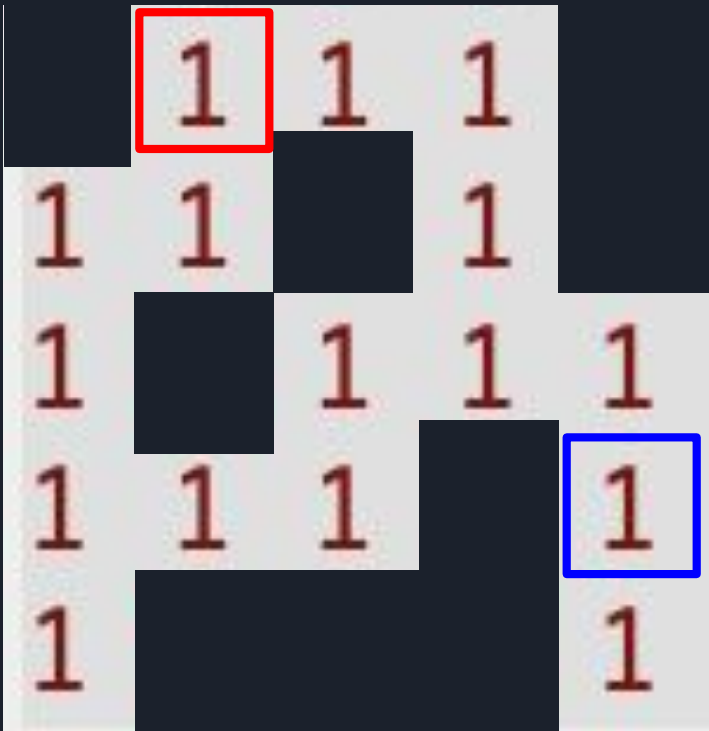
輸入第一行有一正整數 $n (2 \leq n \leq 8)$

第二行有四個非負整數，分別為座標 i_0 、 j_0 、 i_1 、 j_1 ($0 \leq i_0, j_0, i_1, j_1 \leq n-1, (i_0, j_0) \neq (i_1, j_1), m[i_0][j_0] = m[i_1][j_1] = 1$)

接下來 n 行，每行有 n 個 0 或 1，為地圖矩陣 m

Output

若有合法路徑，輸出一個正整數，為最短路徑所需的步數；否則輸出 -1





```
// {{i_c, j_c}, length}
queue<pair<pair<int, int>, int>> q;
q.emplace(x: make_pair(x: [>>] i0, y: [>>] j0), y: 0);

while (!q.empty()) {
    pair<pair<int, int>, int> curr = q.front();
    q.pop();

    if (curr.first.first < 0 || curr.first.first >= n || curr.first.second < 0 || curr.first.second >= n) { // 出界
        continue;
    }

    if (m[curr.first.first][curr.first.second] == 0) { // 樹林或找過了
        continue;
    }

    if (curr.first.first == i1 && curr.first.second == j1) { // 找到了
        cout << curr.second << '\n';
        return 0;
    }

    m[curr.first.first][curr.first.second] = 0; // 找過不要再找一遍，避免繞圈圈

    q.emplace(x: make_pair(x: curr.first.first + 1, y: [>>] curr.first.second), y: curr.second + 1);
    q.emplace(x: make_pair(x: curr.first.first - 1, y: [>>] curr.first.second), y: curr.second + 1);
    q.emplace(x: make_pair(x: [>>] curr.first.first, y: curr.first.second + 1), y: curr.second + 1);
    q.emplace(x: make_pair(x: [>>] curr.first.first, y: curr.first.second - 1), y: curr.second + 1);
}
```